

Guía de Desarrollo — Programación de Sockets en Python, .NET Core y NodeJS

Objetivo de la guía

Esta guía tiene como finalidad orientar el desarrollo de ejemplos de comunicación cliente-servidor mediante **sockets**, utilizando los protocolos **UDP** y **TCP** en tres lenguajes de programación: **Python**, **C# con .NET Core** y **NodeJS**.

La propuesta está pensada para alumnos universitarios de Redes de Datos y utiliza **Visual Studio Code** como entorno de desarrollo común para todos los lenguajes. La guía incluye preparación del entorno, estructura de carpetas, nombres de archivos y librerías necesarias para cada tecnología.

1. Entorno general de trabajo

1.1. Instalación de Visual Studio Code

Se recomienda utilizar **Visual Studio Code** por ser un editor liviano, multiplataforma y adecuado para trabajar con varios lenguajes dentro de una misma carpeta de proyecto.

Descargar Visual Studio Code desde:

<https://code.visualstudio.com/>

Luego de instalarlo, abrir una terminal y verificar que el comando **code** esté disponible:

```
code --version
```

Si el comando no funciona, abrir Visual Studio Code y ejecutar:

Ctrl + Shift + P

Luego buscar y seleccionar:

Shell Command: Install 'code' command in PATH

En Windows, si esta opción no aparece, se puede abrir la carpeta manualmente desde Visual Studio Code usando:

File > Open Folder

1.2. Carpeta general del trabajo práctico

Crear una carpeta principal para todos los ejemplos:

```
mkdir sockets-tp
```

```
cd sockets-tp
```

```
code .
```

La estructura general sugerida será:

```
sockets-tp/
```

```
|
```

```
|— python/
```

```
| |— udp/
```

```
| | |— udp_server.py
```

```
| | └─ udp_client.py
```

```
| |
```

```
| └─ tcp/
```

```
| |— tcp_server.py
```

```
| └─ tcp_client.py
```

```
|
```

```
|— dotnet/
```

```
|   ├── UdpServer/
|   ├── UdpClient/
|   ├── TcpServer/
|   └── TcpClient/
|
└── nodejs/
    ├── udp/
    |   ├── udp_server.js
    |   └── udp_client.js
    |
    └── tcp/
        ├── tcp_server.js
        └── tcp_client.js
```

Esta organización permite separar claramente los ejemplos por lenguaje y por protocolo.

2. Desarrollo en Python

2.1. Preparación del entorno Python

Python incluye una biblioteca estándar llamada `socket`, por lo tanto no es necesario instalar librerías externas para estos ejemplos básicos.

Se recomienda instalar **Python 3.11 o superior**.

Verificar la instalación:

```
python --version
```

O, según la configuración del sistema:

```
python3 --version
```

También se recomienda verificar el gestor de paquetes **pip**:

```
pip --version
```

En Visual Studio Code se recomienda instalar la extensión:

Python

Extensión recomendada:

Python - Microsoft

Esta extensión permite autocompletado, ejecución de scripts, selección de intérprete y depuración.

2.2. Librerías necesarias en Python

Para los ejemplos básicos de UDP y TCP no se debe instalar ninguna librería externa, ya que se utiliza:

```
import socket
```

La librería **socket** viene incluida con Python.

Opcionalmente, para proyectos más avanzados se podrían usar:

```
pip install python-dotenv
```

Esto permitiría manejar configuraciones externas, como IP, puerto o modo de ejecución, mediante archivos **.env**. Para el trabajo inicial no es obligatorio.

2.3. Estructura de carpetas en Python

Desde la carpeta principal `sockets-tp`, crear la siguiente estructura:

```
mkdir -p python/udp
```

```
mkdir -p python/tcp
```

En Windows PowerShell, si `mkdir -p` no funciona, usar:

```
mkdir python
```

```
mkdir python\udp
```

```
mkdir python\tcp
```

La estructura esperada será:

```
python/
```

```
|
```

```
|— udp/
```

```
| |— udp_server.py
```

```
| |— udp_client.py
```

```
|
```

```
|— tcp/
```

```
|— tcp_server.py
```

```
|— tcp_client.py
```

2.4. Archivos del proyecto Python

Servidor UDP

Archivo:

`python/udp/udp_server.py`

Este archivo contendrá el servidor que escucha mensajes UDP en un puerto determinado.

Cliente UDP

Archivo:

`python/udp/udp_client.py`

Este archivo contendrá el cliente que envía mensajes UDP al servidor.

Servidor TCP

Archivo:

`python/tcp/tcp_server.py`

Este archivo contendrá el servidor TCP que espera una conexión entrante.

Cliente TCP

Archivo:

`python/tcp/tcp_client.py`

Este archivo contendrá el cliente TCP que se conecta al servidor.

2.5. Ejecución desde Visual Studio Code

Abrir una terminal integrada en VS Code:

Terminal > New Terminal

Para ejecutar el servidor UDP:

```
cd python/udp
```

```
python udp_server.py
```

En otra terminal, ejecutar el cliente UDP:

```
cd python/udp
```

```
python udp_client.py
```

Para ejecutar el servidor TCP:

```
cd python/tcp
```

```
python tcp_server.py
```

En otra terminal, ejecutar el cliente TCP:

```
cd python/tcp
```

```
python tcp_client.py
```

El servidor debe ejecutarse antes que el cliente, porque queda esperando conexiones o mensajes.

3. Desarrollo en C# con .NET Core

3.1. Preparación del entorno .NET Core

Para desarrollar sockets en C# se utilizará **.NET SDK**, que permite crear, compilar y ejecutar aplicaciones de consola multiplataforma.

Se recomienda instalar **.NET 8 SDK o superior**.

Verificar la instalación:

```
dotnet --version
```

También se puede listar la información completa del entorno:

`dotnet --info`

En Visual Studio Code se recomienda instalar alguna de las siguientes extensiones:

C# Dev Kit - Microsoft

O también:

C# - Microsoft

La extensión permite autocompletado, depuración, navegación entre clases y reconocimiento de proyectos `.csproj`.

3.2. Librerías necesarias en .NET Core

Para los ejemplos básicos de sockets no es necesario instalar paquetes externos mediante NuGet. Se utilizan namespaces incluidos en .NET:

`using System;`

`using System.Net;`

`using System.Net.Sockets;`

`using System.Text;`

Los principales componentes utilizados serán:

UdpClient

TcpClient

TcpListener

NetworkStream

IPEndPoint

IPAddress

Encoding

Opcionalmente, para proyectos más avanzados se podrían instalar paquetes como:

```
dotnet add package Microsoft.Extensions.Configuration
```

O:

```
dotnet add package Microsoft.Extensions.Configuration.Json
```

Esto permitiría manejar archivos `appsettings.json`. Para los ejemplos iniciales no es obligatorio.

3.3. Estructura de carpetas en .NET Core

Desde la carpeta principal `sockets-tp`, crear una carpeta para los proyectos .NET:

```
mkdir dotnet
```

```
cd dotnet
```

Crear cuatro aplicaciones de consola independientes:

```
dotnet new console -n UdpServer
```

```
dotnet new console -n UdpClient
```

```
dotnet new console -n TcpServer
```

```
dotnet new console -n TcpClient
```

La estructura esperada será:

```
dotnet/
```

```
|
```

```
|— UdpServer/
```

```
| |— Program.cs
```

| |— UdpServer.csproj

| |— bin/

| |— obj/

|

|— UdpClient/

| |— Program.cs

| |— UdpClient.csproj

| |— bin/

| |— obj/

|

|— TcpServer/

| |— Program.cs

| |— TcpServer.csproj

| |— bin/

| |— obj/

|

|— TcpClient/

|— Program.cs

|— TcpClient.csproj

|— bin/

|— obj/

Los archivos `bin/` y `obj/` son generados automáticamente por .NET durante la compilación y ejecución.

3.4. Archivos del proyecto .NET Core

Servidor UDP

Proyecto:

`dotnet/UdpServer/`

Archivo principal:

`dotnet/UdpServer/Program.cs`

En este archivo se implementa el servidor UDP que escucha en un puerto determinado.

Cliente UDP

Proyecto:

`dotnet/UdpClient/`

Archivo principal:

`dotnet/UdpClient/Program.cs`

En este archivo se implementa el cliente UDP que envía un mensaje al servidor.

Servidor TCP

Proyecto:

`dotnet/TcpServer/`

Archivo principal:

`dotnet/TcpServer/Program.cs`

En este archivo se implementa el servidor TCP que espera una conexión entrante.

Cliente TCP

Proyecto:

dotnet/TcpClient/

Archivo principal:

dotnet/TcpClient/Program.cs

En este archivo se implementa el cliente TCP que se conecta al servidor.

3.5. Ejecución desde Visual Studio Code

Desde la terminal integrada de VS Code, ejecutar primero el servidor y luego el cliente correspondiente.

Ejecutar servidor UDP

```
cd dotnet/UdpServer
```

```
dotnet run
```

Ejecutar cliente UDP

Abrir otra terminal:

```
cd dotnet/UdpClient
```

```
dotnet run
```

Ejecutar servidor TCP

```
cd dotnet/TcpServer
```

```
dotnet run
```

Ejecutar cliente TCP

Abrir otra terminal:

```
cd dotnet/TcpClient
```

dotnet run

3.6. Compilación en .NET Core

Para compilar sin ejecutar:

dotnet build

Para limpiar archivos generados:

dotnet clean

Para ejecutar una aplicación:

dotnet run

4. Desarrollo en NodeJS

4.1. Preparación del entorno NodeJS

NodeJS permite desarrollar aplicaciones de red utilizando módulos nativos para TCP y UDP. Es especialmente útil para comprender programación orientada a eventos y aplicaciones en tiempo real.

Instalar **NodeJS LTS** desde:

<https://nodejs.org/>

Verificar la instalación:

node --version

Verificar el gestor de paquetes NPM:

npm --version

En Visual Studio Code se recomienda instalar extensiones como:

JavaScript and TypeScript Nightly

Opcionalmente:

ESLint

Prettier

Para los ejemplos básicos, Visual Studio Code ya ofrece buen soporte para JavaScript sin instalar extensiones adicionales.

4.2. Librerías necesarias en NodeJS

Para los ejemplos básicos no se requiere instalar librerías externas. NodeJS incluye módulos nativos para sockets:

Para TCP

```
const net = require('net');
```

Para UDP

```
const dgram = require('dgram');
```

El módulo `net` permite crear servidores y clientes TCP.

El módulo `dgram` permite trabajar con datagramas UDP.

Opcionalmente, para proyectos más avanzados se podrían instalar:

```
npm install dotenv
```

Esto permitiría manejar configuraciones externas mediante archivos `.env`. Para la práctica inicial no es obligatorio.

4.3. Estructura de carpetas en NodeJS

Desde la carpeta principal `sockets-tp`, crear la siguiente estructura:

```
mkdir -p nodejs/udp
```

```
mkdir -p nodejs/tcp
```

En Windows PowerShell, si `mkdir -p` no funciona, usar:

```
mkdir nodejs
```

```
mkdir nodejs\udp
```

```
mkdir nodejs\tcp
```

La estructura esperada será:

```
nodejs/
```

```
|
```

```
|— udp/
```

```
| |— udp_server.js
```

```
| |— udp_client.js
```

```
|
```

```
|— tcp/
```

```
|— tcp_server.js
```

```
|— tcp_client.js
```

4.4. Inicialización opcional de proyecto NodeJS

Aunque los ejemplos pueden ejecutarse directamente con `node`, se recomienda inicializar un proyecto NodeJS para mantener una estructura profesional.

Desde la carpeta `nodejs`:

```
cd nodejs
```

npm init -y

Esto genera un archivo:

package.json

La estructura puede quedar así:

nodejs/

|

|— package.json

|

|— udp/

| |— udp_server.js

| |— udp_client.js

|

|— tcp/

|— tcp_server.js

|— tcp_client.js

4.5. Archivos del proyecto NodeJS

Servidor UDP

Archivo:

nodejs/udp/udp_server.js

Este archivo contendrá el servidor UDP que escucha datagramas en un puerto.

Cliente UDP

Archivo:

`nodejs/udp/udp_client.js`

Este archivo contendrá el cliente UDP que envía un mensaje al servidor.

Servidor TCP

Archivo:

`nodejs/tcp/tcp_server.js`

Este archivo contendrá el servidor TCP basado en eventos.

Cliente TCP

Archivo:

`nodejs/tcp/tcp_client.js`

Este archivo contendrá el cliente TCP que se conecta al servidor.

4.6. Ejecución desde Visual Studio Code

Ejecutar servidor UDP

```
cd nodejs/udp
```

```
node udp_server.js
```

Ejecutar cliente UDP

En otra terminal:

```
cd nodejs/udp
```

```
node udp_client.js
```

Ejecutar servidor TCP

```
cd nodejs/tcp
```

```
node tcp_server.js
```

Ejecutar cliente TCP

En otra terminal:

```
cd nodejs/tcp
```

```
node tcp_client.js
```

5. Recomendaciones para trabajar en Visual Studio Code

5.1. Abrir siempre la carpeta principal

Se recomienda abrir la carpeta completa del trabajo:

```
cd sockets-tp
```

```
code .
```

Esto permite ver los proyectos de Python, .NET Core y NodeJS en un único explorador de archivos.

5.2. Usar terminales separadas

Como los ejemplos cliente-servidor requieren ejecutar más de un programa al mismo tiempo, se recomienda abrir varias terminales integradas:

Terminal > New Terminal

Una terminal debe quedar ejecutando el servidor y otra terminal debe ejecutar el cliente.

5.3. Ejecutar siempre primero el servidor

En todos los ejemplos se debe iniciar primero el servidor.

Orden recomendado:

1. Ejecutar servidor UDP o TCP.
 2. Verificar que quedó escuchando.
 3. Ejecutar cliente UDP o TCP.
 4. Observar mensajes enviados y recibidos.
-

5.4. Uso de localhost

Para las primeras pruebas se utilizará:

localhost

O también:

127.0.0.1

Esto significa que cliente y servidor se ejecutan en la misma computadora.

5.5. Prueba entre dos computadoras

Para probar en una red real, el servidor debe ejecutarse en una computadora y el cliente en otra.

En ese caso, reemplazar:

localhost

Por la dirección IP de la computadora donde corre el servidor.

Ejemplo:

192.168.1.25

En Windows se puede consultar la IP con:

ipconfig

En Linux o macOS:

ip addr

O:

ifconfig

6. Puertos utilizados en los ejemplos

Se recomienda usar puertos distintos para evitar conflictos:

UDP: 5005

TCP: 6000

Si el puerto está ocupado, se puede cambiar por otro valor, por ejemplo:

UDP: 5010

TCP: 6010

Debe recordarse cambiar el puerto tanto en el servidor como en el cliente.

7. Problemas frecuentes

7.1. El cliente no se conecta

Verificar que el servidor esté ejecutándose antes que el cliente.

7.2. El puerto está ocupado

Cambiar el puerto o cerrar el proceso que lo está utilizando.

7.3. Firewall bloqueando la conexión

En pruebas entre computadoras, el firewall puede bloquear conexiones entrantes. Se debe permitir el acceso de Python, .NET o NodeJS en la red local.

7.4. Error con localhost

Probar usando:

127.0.0.1

7.5. Problemas entre dos computadoras

Verificar:

- Que ambas computadoras estén en la misma red.
 - Que se pueda hacer ping entre ellas.
 - Que el servidor use `IPAddress.Any` o `0.0.0.0` cuando corresponda.
 - Que el cliente apunte a la IP correcta del servidor.
-

8. Resumen de librerías por lenguaje

Lenguaje	Librería / Módulo	Instalación requerida	Uso principal
Python	<code>socket</code>	No	TCP y UDP
C# .NET Core	<code>System.Net.Sockets</code>	No	TCP y UDP
NodeJS	<code>net</code>	No	TCP
NodeJS	<code>dgram</code>	No	UDP
Python opcional	<code>python-dotenv</code>	Sí	Configuración externa
.NET opcional	<code>Microsoft.Extensions.Configuration.Json</code>	Sí	Configuración con JSON
NodeJS opcional	<code>dotenv</code>	Sí	Configuración externa

9. Comandos principales por tecnología

Python

`python --version`

`python archivo.py`

.NET Core

`dotnet --version`

`dotnet new console -n NombreProyecto`

`dotnet run`

`dotnet build`

`dotnet clean`

NodeJS

`node --version`

`npm --version`

`npm init -y`

`node archivo.js`

10. Cierre

Esta guía permite preparar un entorno de desarrollo común para implementar aplicaciones de sockets en Python, .NET Core y NodeJS. La comparación entre lenguajes permite observar que los conceptos fundamentales de redes —IP, puerto, socket, cliente, servidor, TCP y UDP— se mantienen constantes, aunque cambie la sintaxis y la forma de trabajo de cada plataforma.

El uso de Visual Studio Code facilita una experiencia integrada, permitiendo trabajar con varios lenguajes, ejecutar terminales simultáneas y organizar los proyectos de forma profesional.